

Motion Detection Cut

Specificatie de Cerinte Software (SRS)

Versiunea 2.0 | Milestone 2

Programare Concurenta si Distribuita

Echipa:

Damian Andrei

Nagy Robert

Groza Paul

Domocos Darian

Istoricul Revizuirilor

Versiune	Data	Autor	Descriere
1.0	Martie 2025	Echipa	SRS initial - Milestone 1
2.0	Mai 2025	Echipa	SRS extins - Milestone 2: client admin, client remote, transfer fisiere, coada de procesare, protocol actualizat

1. Introducere

1.1 Scop

Acest document descrie specificatiile de cerinte software pentru sistemul Motion Detection Cut - o aplicatie Client-Server care realizeaza detectarea miscarii in fisiere video folosind OpenCV si paralelism bazat pe procese. Versiunea 2.0 extinde SRS-ul de la Milestone 1 pentru a acoperi scopul complet al Milestone 2: doua tipuri distincte de clienti, un protocol de transfer de fisiere bidirectional, o coada de procesare si operatii extinse de server.

1.2 Context

Aplicatia este dezvoltata in C/C++ si integreaza concepte din mai multe domenii:

- Comunicatie in retea (sockets TCP)
- Programare concurenta (procese multiple cu fork())
- Procesare video (OpenCV)
- Configurare externa (libconfig)

Sistemul este destinat utilizarii academice pentru demonstrarea integrarii acestor concepte in cadrul cursului de Programare Concurenta si Distribuita.

1.3 Definitii si Acronime

Termen	Definitie
Motion Detection	Identificarea diferentelor dintre cadre video consecutive prin metoda diferentei de cadre
Worker	Proces copil creat prin fork() care proceseaza un segment de cadre video
Chunk	Subset contiguu de cadre video atribuit unui singur proces worker
Cadru fara miscare	Cadru in care diferenta pixel cu pixel fata de cadrul anterior este sub pragul configurat
Job	O singura cerere de procesare video urmarita de server de la trimitere pana la finalizare
Client Admin (UX)	Client TCP usor utilizat pentru intretinerea serverului; fara transfer de fisiere
Client Remote (IN)	Client TCP complet care suporta incarcarea video, procesarea si descarcarea rezultatelor

Termen	Definitie
IPC	Comunicare inter-procese (Inter-Process Communication) - implementata prin pipe-uri UNIX
SRS	Specificatie de Cerinte Software (acest document)
SDD	Document de Design Software (livrabil separat)

1.4 Referinte

- IEEE Std 830-1998: Practica Recomandata pentru Specificatiile de Cerinte Software
- POSIX.1-2017 - fork(), pipe(), waitpid(), getopt()
- Documentatie OpenCV 4.x - VideoCapture, Mat
- Documentatie libconfig - <https://hyperrealm.github.io/libconfig/>
- SRS Milestone 1 v1.0 (prezentul document il inlocuieste)

2. Descriere Generala

2.1 Arhitectura

Sistemul urmeaza o arhitectura de server TCP cu mai multi clienti. Procesul server accepta conexiuni de la doua tipuri distincte de clienti si dispatcuiesc cererile catre procese worker copil. Toata concurenta pe partea de server foloseste exclusiv fork()/waitpid(). Nu se utilizeaza fire de executie (thread-uri) nicaieri in sistem.

Structura componentelor la nivel inalt:

```

[Client Admin] --TCP--> [Proces Server] --fork()--> [Worker 1]
[Client Remote] --TCP-->                |         --fork()--> [Worker 2]
                                   |         --fork()--> [Worker N]
                                   |
                                   [Coadă Joburi]
                                   [Agregator]

```

2.2 Tipuri de Clienti

Sistemul suporta doua categorii de clienti, fiecare cu un rol distinct si un set permis de operatii:

Proprietate	Client Admin (UX)	Client Remote (IN)
Numele binarului	adminclient	remoteclient
Scop	Intretinere si monitorizare server	Procesare video si transfer fisiere
Transfer fisiere	INTERZIS	Obligatori (bidirectional pentru nota maxima)
Parsare CLI	getopt() obligatoriu	getopt() obligatoriu
Operatii	STATUS, STATS, LIST_JOBS, SHUTDOWN, GET_CONFIG	PROCESS_VIDEO, UPLOAD_VIDEO, DOWNLOAD_RESULT,

Proprietate	Client Admin (UX)	Client Remote (IN)
		GET_RESULT, GET_PROGRESS, DELETE_RESULT
Prefix mesaj	ADM	REQ / UPLOAD

2.3 Fluxul de Executie

1. Clientul se conecteaza la server prin TCP
2. Serverul citeste prima linie de protocol si dispatcuieste dupa prefix (ADM|, REQ|, UPLOAD|)
3. Pentru cereri admin: serverul executa operatia si returneaza raspunsul sincron
4. Pentru cereri remote: serverul valideaza cererea, pune jobul in coada, forkeaza un worker
5. Worker-ul proceseaza cadrele video si raporteaza rezultatele pariale catre parinte prin pipe
6. Parintele agrega rezultatele, actualizeaza statusul si progresul jobului in coada
7. Serverul trimite raspunsul final catre client
8. Clientul primeste si afiseaza rezultatul; pentru DOWNLOAD_RESULT, salveaza fisierul pe disc

3. Cerinte Functionale

3.1 Cerinte Server

3.1.1 Gestionarea Conexiunilor

- Serverul TREBUIE sa accepte mai multe conexiuni TCP secventiale de la clienti
- Fiecare conexiune acceptata TREBUIE gestionata intr-un proces copil forkat
- Serverul TREBUIE sa distinga prefixele de mesaje ADM|, REQ| si UPLOAD|
- Prefixele necunoscute TREBUIE sa primeasca ERR|cerere_necunoscuta si conexiunea este inchisa
- Serverul TREBUIE sa gestioneze comanda admin SHUTDOWN elegant: termina joburile in curs, apoi iese

3.1.2 Coada de Procesare

- Serverul TREBUIE sa mentina o coada de joburi sub forma de lista inlantuita in procesul parinte
- Fiecare intrare din coada TREBUIE sa stocheze: job_id, video_path, chunk_size, threshold, status, progress
- Valorile statusului unui job: QUEUED (in asteptare), RUNNING (in executie), DONE (finalizat), ERROR (eroare)
- Progresul este un intreg de la 0 la 100 inclusiv
- Procesul parinte pune joburi in coada si forkeaza worker-i pentru a le consuma
- Worker-ii raporteaza progresul inapoi catre parinte prin IPC cu pipe

3.1.3 Gestionarea Proceselor Worker

- Worker-ii TREBUIE creati exclusiv cu fork() - fara thread-uri
- Fiecare worker proceseaza un chunk de cadre video folosind OpenCV
- Worker-ii comunica rezultatele catre parinte exclusiv prin pipe
- Parintele TREBUIE sa recupereze toate procesele copil cu waitpid() pentru a preveni procesele zombie
- Numarul maxim de worker-i concurenti este citit din server.cfg (parametrul max_workers)

3.2 Operatii Client Admin

Toate operatiile folosesc formatul de mesaj: ADM|<COMANDA>

Operatie	Cerere	Raspuns Server	Descriere
STATUS	ADM STATUS	RES OK <sir_status>	Returneaza starea de functionare a serverului
STATS	ADM STATS	RES <joburi_finalizate> <cadre_analizate>	Returneaza statistici de procesare
LIST_JOBS	ADM LIST_JOBS	JOB <id> <status> <progres>\n...\nEND	Listeaza toate joburile din coada
GET_CONFIG	ADM GET_CONFIG	RES <max_workers> <chunk_size> <threshold>	Returneaza valorile configuratiei incarcate
SHUTDOWN	ADM SHUTDOWN	RES OK	Initiaza oprirea eleganta a serverului

3.3 Operatii Client Remote

Operatiile standard folosesc formatul: REQ|<OPERATIE>|<arg1>|<arg2>|...

Operatiile de incarcare fisiere folosesc prefixul UPLOAD| (vezi Sectiunea 4).

Operatie	Format Cerere	Raspuns Server	Transfer Fisier
PROCESS_VIDEO	REQ PROCESS_VIDEO <cale> <chunk_size> <threshold>	RES JOB_ID <id>	Niciunul - fisier local pe server
UPLOAD_VIDEO	UPLOAD <fisier> <dimensiune> apoi date binare	RES UPLOAD_OK apoi RES JOB_ID <id>	Client -> Server
DOWNLOAD_RESULT	REQ DOWNLOAD_RESULT <fisier>	DOWNLOAD <fisier> <dim> apoi date binare	Server -> Client

Operatie	Format Cerere	Raspuns Server	Transfer Fisier
GET_RESULT	REQ GET_RESULT <job_id>	RES <continut_text>	Niciunul
GET_PROGRESS	REQ GET_PROGRESS <job_id>	RES PROGRESS <0-100>	Niciunul
DELETE_RESULT	REQ DELETE_RESULT <job_id>	RES OK sau ERR NOT_FOUND	Niciunul

4. Protocolul de Comunicare

4.1 Transport

- Toata comunicarea foloseste sockets TCP
- Toate mesajele text sunt terminate cu newline (\n)
- Mesajele text folosesc bara verticala (|) ca separator de campuri
- Datele binare ale fisierelor sunt trimise brut dupa un schimb de handshake (vezi Sectiunea 4.3)
- Toate apelurile send()/recv() TREBUIE infasurate in bucle de reincercare pentru a gestiona I/O partial

4.2 Tipuri de Mesaje

Prefix	Directie	Semnificatie
ADM	Client -> Server	Comanda admin
REQ	Client -> Server	Cerere de operatie client remote
UPLOAD	Client -> Server	Initiaza o incarcare de fisier (urmata de date binare)
DOWNLOAD	Server -> Client	Initiaza o descarcare de fisier (urmata de date binare)
READY	Server -> Client sau Client -> Server	Confirmare: gata de a primi date binare
RES	Server -> Client	Raspuns de succes cu campuri optionale de date
ERR	Server -> Client	Raspuns de eroare cu sirul motivului
JOB	Server -> Client	Inregistrare individuala de job (folosita in raspunsul LIST_JOBS)
END	Server -> Client	Termina un raspuns pe mai multe linii

4.3 Protocolul de Transfer Fisiere

4.3.1 Flux Incarcare (Client catre Server)

```
Client -> Server:  UPLOAD|<nume_fisier>|<dimensiune_fisier>\n
Server -> Client:  READY\n
```

```
Client -> Server: <date binare brute, exact <dimensiune_fisier> octeti>
Server -> Client: RES|UPLOAD_OK\n
```

4.3.2 Flux Descarcare (Server catre Client)

```
Client -> Server: REQ|DOWNLOAD_RESULT|<nume_fisier>\n
Server -> Client: DOWNLOAD|<nume_fisier>|<dimensiune_fisier>\n
Client -> Server: READY\n
Server -> Client: <date binare brute, exact <dimensiune_fisier> octeti>
Server -> Client: RES|DOWNLOAD_DONE\n
```

4.3.3 Reguli de Siguranta pentru Transferul de Fisiere

- Dimensiunea maxima acceptata a fisierului: 500 MB (aplicata atat la incarcare cat si la descarcare)
- Numele fisierelor TREBUIE sanitizate: se resping orice nume care contine .. sau /
- Toata transmisia datelor binare TREBUIE sa foloseasca send()/recv() in bucla (functii ajutatoare send_all / recv_all)
- Fisierele incarcate sunt stocate in directorul uploads/ de pe server
- Fisierele cu rezultate sunt stocate in directorul results/ de pe server

4.4 Gestionarea Erorilor

Cod Eroare	Semnificatie
ERR cerere_necunoscuta	Prima linie nu s-a potrivit cu niciun prefix cunoscut
ERR comanda_necunoscuta	Comanda ADM nu este recunoscuta
ERR operatie_necunoscuta	Operatia REQ nu este recunoscuta
ERR argumente_invalide	Argumente obligatorii lipsa sau malformate
ERR NOT_FOUND	ID-ul jobului sau fisierul referentiat nu exista
ERR FILE_TOO_LARGE	Fisierul incarcat depaseste dimensiunea maxima
ERR upload_esuat	Serverul nu a putut scrie fisierul incarcat
ERR procesare_esuata	Procesul worker a returnat o eroare

5. Cerinte Non-Functionale

5.1 Corectitudine

- Serverul TREBUIE sa returneze numarul corect de cadre fara miscare pentru toate videoclipurile procesate
- Progresul jobului raportat prin GET_PROGRESS TREBUIE sa reflecte progresul real al worker-ului, nu o estimare
- Toate mesajele de protocol TREBUIE parsate complet inainte de a actiona pe baza lor

5.2 Siguranta si Robustete

- Nicio functie nesigura din biblioteca standard C: gets(), sprintf(), strcpy() sau strcat() nelimitat sunt interzise
- Toate alocarile de memorie heap TREBUIE eliberate inainte ca procesul care le-a alocat sa iasa
- Toti descriptorii de fisiere si socket-urile TREBUIE inchise inainte ca procesul proprietar sa iasa
- Toate procesele copil forked TREBUIE recuperate cu waitpid() - niciun proces zombie nu este permis
- Serverul NU TREBUIE sa se blocheze la primirea unui input malformat sau prea mare de la orice client
- Serverul TREBUIE sa valideze toate campurile primite din retea inainte de utilizare

5.3 Calitatea Codului

- Toate fisierele sursa TREBUIE sa se compileze cu: -Wall -Wextra -Werror - zero avertismente permise
- Toate fisierele sursa TREBUIE sa treaca clang-tidy cu configuratia stricta a proiectului
- Niciun comportament nedefinit - verificari defensive pentru toate dereferencierile de pointeri si operatiile aritmetice
- Toata parsarea argumentelor CLI TREBUIE sa foloseasca getopt() - indexarea manuala a argv este interzisa
- Utilizarea invalida a CLI TREBUIE sa afiseze o linie de utilizare si sa iasa cu codul de retur 1

5.4 Portabilitate

- Implementarea tinteste Linux (Ubuntu 22.04+)
- Nicio extensie specifica platformei dincolo de POSIX.1-2017 si API-urile de socket Linux
- Sistemul de build este GNU Make; un singur 'make all' TREBUIE sa construiasca toate cele trei binare

5.5 Performanta

- Serverul TREBUIE sa proceseze fisiere de pana la cateva sute de MB fara a se bloca
- Numarul de worker-i este configurabil prin server.cfg (parametrul max_workers)
- Sistemul nu trebuie sa gestioneze conexiuni concurente simultan in acest milestone (bucla accept secventiala)

6. Configurare

6.1 Formatul server.cfg

Serverul citeste toti parametrii de rulare dintr-un fisier de configurare libconfig specificat prin -f in CLI.


```

server:
{
    host          = "127.0.0.1";    # Adresa IP de legare
    port          = 9090;           # Portul TCP de ascultare
    max_workers   = 4;              # Numarul maxim de procese worker
concurrente
    chunk_size    = 100;            # Numarul implicit de cadre per chunk
    threshold     = 25;            # Pragul implicit pentru detectarea
miscarii
    upload_dir    = "uploads/";     # Director pentru fisierele video
primate
    result_dir    = "results/";     # Director pentru rezultatele
procesarii
    max_file_mb   = 500;            # Dimensiunea maxima acceptata a
fisiierului in MB
};

```

6.2 Referinta CLI

Binar	Utilizare	Optiuni Obligatorii	Note
server	./server -f <fisier_config>	-f calea catre config	Iese cu codul 1 daca fisierul de config lipseste sau nu este citibil
adminclient	./adminclient -h <host> -p <port> -c <comanda>	-h, -p, -c toate obligatorii	Comenzi valide: STATUS STATS LIST_JOBS SHUTDOWN GET_CONFIG
remoteclient	./remoteclient -h <host> -p <port> -o <operatie> [-f <fisier>] [-j <job_id>] [-t <threshold>] [-c <chunk>]	-h, -p, -o obligatorii; celelalte depind de operatie	Foloseste getopt(); afiseaza utilizare si iese cu 1 la eroare

7. Matricea de Acoperire a Operatiilor

Aceasta sectiune mapeaza operatiile implementate la cerinta Milestone 2 de cel putin 70% din operatiile definite in SRS.

Operatie	Tip Client	Implementa ta	Transfer Fisier	Prioritat e
STATUS	Admin (UX)	Da	Niciunul	Mare
STATS	Admin (UX)	Da	Niciunul	Mare

Operatie	Tip Client	Implementa ta	Transfer Fisier	Prioritat e
LIST_JOBS	Admin (UX)	Da	Niciunul	Mare
GET_CONFIG	Admin (UX)	Da	Niciunul	Mare
SHUTDOWN	Admin (UX)	Da	Niciunul	Mare
PROCESS_VIDEO	Remote (IN)	Da	Niciunul	Mare
UPLOAD_VIDEO	Remote (IN)	Da	Client -> Server	Mare
DOWNLOAD_RESULT	Remote (IN)	Da	Server -> Client	Mare
GET_RESULT	Remote (IN)	Da	Niciunul	Mare
GET_PROGRESS	Remote (IN)	Da	Niciunul	Medie
DELETE_RESULT	Remote (IN)	Da	Niciunul	Medie

Total: 11 operatii definite, 11 implementate = 100% acoperire (depaseste cerinta minima de 70%).

8. Modelul de Procese

8.1 Ierarhia Proceselor

```

[Proces Parinte Server]
|
|-- bucla accept()
|-- Coada Joburi (lista inlantuita)
|-- fork() --> [Handler Conexiune Copil]
|
|               |-- parseaza cererea
|               |-- pune jobul in coada
|               |-- fork() --> [Proces Worker]
|               |               |-- procesare OpenCV
|               |               |-- scrie rezultat in
pipe
|-- waitpid() toate procesele copil

```

8.2 Design IPC

- Comunicarea parinte-worker: pipe anonim UNIX (pipe())
- Worker-ii scriu rezultatele lor parțiale (numar cadre fara miscare) la capatul de scriere al pipe-ului
- Parintele citeste de la capatul de citire si agrega rezultatele
- Worker-ii TREBUIE sa inchida imediat capetele de pipe neutilizate dupa fork()
- Parintele TREBUIE sa inchida capetele de scriere ale tuturor pipe-urilor pe care nu le detine

8.3 Constrangeri

- Multithreading-ul este strict interzis - se folosesc exclusiv fork() si procese
- Nicio memorie partajata - tot IPC-ul se face prin pipe-uri
- Nicio coada de mesaje, semafoare sau semnale pentru operatiunea normala (SIGCHLD este acceptabil pentru recuperarea copiilor)

9. Transferul de Fisiere si Impactul Asupra Notarii

Nivel	Descriere	Interval Nota
Nivel 1 - Minim	Fara transfer de fisiere. Serverul proceseaza doar fisiere locale de pe server.	Cel mult 70%
Nivel 2 - Intermediar	Transfer unidirectional: clientul incarca videoclipul pe server. Serverul il stocheaza si proceseaza.	70% - 85%
Nivel 3 - Credit complet	Transfer bidirectional: clientul incarca videoclipul SI descarca rezultatele. I/O in bucla sigura. Suporta fisiere de cateva sute de MB.	85% - 100%

Aceasta implementare tinteaza Nivelul 3 (transfer bidirectional de fisiere) pentru nota maxima.

10. Cerinte de Testare

10.1 Teste Functionale

- Fiecare operatie admin trebuie verificata individual pe un server care ruleaza
- Fiecare operatie remote trebuie verificata individual, inclusiv cazurile limita
- Incarcarea fisierelor trebuie verificata pentru corectitudine: numarul de octeti al fisierului primit trebuie sa corespunda cu cel trimis
- Descarcarea fisierelor trebuie verificata: fisierul descarcat trebuie sa fie identic cu cel de pe server

10.2 Teste de Robustete

- Serverul nu trebuie sa se blocheze la primirea unui mesaj malformat (campuri lipsa, prefix gresit)
- Serverul trebuie sa respinga fisierele mai mari decat max_file_mb cu ERR|FILE_TOO_LARGE
- Serverul trebuie sa respinga numele de fisier care contin .. sau /
- Trimiterea GET_PROGRESS pentru un job_id inexistent trebuie sa returneze ERR|NOT_FOUND

10.3 Teste de Scurgeri de Resurse

- Dupa 10 conexiuni secventiale de la clienti, se verifica absenta proceselor zombie cu: `ps aux | grep Z`
- Dupa procesare, se verifica inchiderea tuturor descriptorilor de fisiere folosind `/proc/<pid>/fd`
- Valgrind sau AddressSanitizer TREBUIE sa raporteze zero scurgeri de memorie la iesirea normala

10.4 Teste de Dimensiune a Fisierelor

- Test cu un fisier video mic (< 10 MB): verificare corectitudine
- Test cu un fisier video mediu (~100 MB): verificare absenta blocajelor sau trunchierilor
- Test cu un fisier video mare (~300-500 MB): verificare transfer si procesare stabile

11. Constrangeri si Limitari

- Niciun multithreading nicaieri in sistem - aceasta este o constrangere stricta a cursului
- Sistemul gestioneaza o singura conexiune de client pe rand in acest milestone (buclo accept secventiala)
- Sistemul este destinat demonstratiei academice, nu utilizarii in productie
- OpenCV este necesar doar pe partea de server; clientii nu se leaga impotriva OpenCV
- libconfig este necesar doar pe partea de server pentru parsarea configuratiei
- Sistemul tinteste Linux/POSIX; compatibilitatea cu macOS nu este obligatorie

12. Glosar

Termen	Definitie
<code>fork()</code>	Apel de sistem POSIX care creaza un nou proces copil prin duplicarea procesului apelant
<code>waitpid()</code>	Apel de sistem POSIX folosit de un proces parinte pentru a astepta si recupera un proces copil
<code>pipe()</code>	Apel de sistem POSIX care creaza un canal de date unidirectional intre doi descriptori de fisiere
<code>getopt()</code>	Functie POSIX pentru parsarea optiunilor liniei de comanda intr-un mod standard
Socket TCP	Endpoint de comunicare fiabil, orientat pe conexiune, folosind Protocolul de Control al Transmisiei
libconfig	Biblioteca C/C++ pentru citirea si scrierea fisierelor de configurare structurate
OpenCV	Biblioteca Open Source de Computer Vision folosita pentru procesarea video si analiza cadrelor

Termen	Definitie
clang-tidy	Instrument de analiza statica C++ folosit pentru aplicarea standardelor de codare si detectarea bug-urilor
Proces zombie	Proces copil care a iesit dar al carui status de iesire nu a fost inca colectat de parinte
POSIX	Interfata Portabila pentru Sistemele de Operare (Portable Operating System Interface) - familie de standarde pentru sisteme de tip Unix